



Symmetric Needham-Schröder

1. $A \rightarrow TTP : A, B, N_a$
2. $TTP \rightarrow A : \{N_a, B, K_{ab}, \{K_{ab}, A\}K_b\}K_a$
3. $A \rightarrow B : \{K_{ab}, A\}K_b$
4. $B \rightarrow A : \{N_b\}K_{ab}$
5. $A \rightarrow B : \{N_b - 1\}K_{ab}$

•ticket

- A che serve N_a ?
- K_{ab} è **chiave di sessione**
- Mutua autentica mediante passi 4 e 5



Replay attack

Def. Spacciare informazione (*chiavi, ...*)
obsoleta, magari violata, come recente

- Supponiamo che C abbia violato una vecchia chiave di sessione K_{ab} che B condivide con A

- ...

3. **C** $\rightarrow$ B : $\{K_{ab}, A\}K_b$ (rispedito
identico)

4. B $\rightarrow$ A : $\{N_b'\}K_{ab}$ (intercettato)

5. **C** $\rightarrow$ B : $\{N_b'-1\}K_{ab}$

- B autenticherebbe A e quindi accetterebbe di usare K_{ab}



Autenticazione vs. Usabilità

- Vogliamo un protocollo sicuro che risolva i limiti del precedente
 - Abbiamo già toccato i limiti delle nonce e utilizzeremo adesso i timestamp
- Vorremmo poterlo usare per accedere in maniera sicura ai servizi della nostra LAN
 - Però ogni accesso a un servizio di rete causerebbe una procedura d'autenticazione

Soluzione: Single Sign-On tramite Kerberos!



Accesso singolo (Single Sign-ON)

Def. Usare unica credenziale di autenticazione per accedere a tutti i servizi

- Soluzione comoda ma poco robusta
 - *Un'unica password per tutto*
- **Kerberos** è un protocollo reale che si occupa di questo problema (e non solo)



Kerberos (fine anni '80)

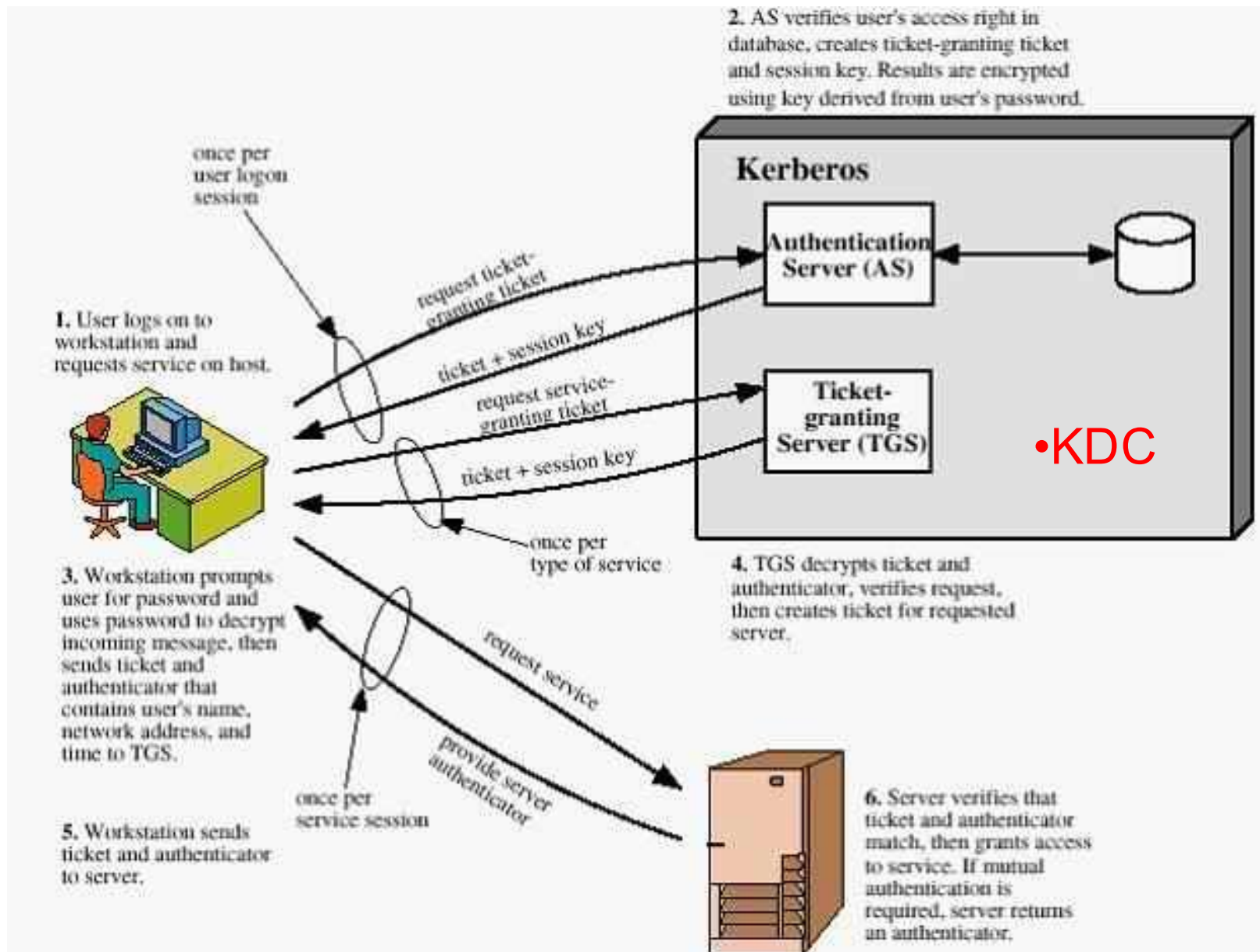
- In mitologia Greca: cane a tre teste guardiano delle porte dell'Ade
- Goal: segretezza, autentica (ad accesso singolo), **temporalità**
 - le chiavi usate hanno validità limitata onde prevenire **replay attack**
- Usa i **timestamp**, che richiedono macchine sincronizzate, contro replay attack





Kerberos in rete







Kerberos – caratteristiche

- 3 fasi: autenticazione, autorizzazione, servizio
- Ultime 2 opzionali e trasparenti all'utente
- Ognuna fornisce credenziali per successiva
 - Fase I fornisce **authKey** e **authTicket** per la II
 - Fase II fornisce **servKey** e **servTicket** per la III
- Ogni tipo di chiave di sessione ha sua durata
- Una authKey può criptare diverse servKey



Kerberos – eventi

- I. AUTENTICAZIONE

1. $A \rightarrow AS : A, TGS, T1$

2. $AS \rightarrow A : \{authK, TGS, Ta, \underbrace{\{A, TGS, authK, Ta\}K_{tgs}}_{\bullet authTicket}\}K_a$

- II. AUTORIZZAZIONE

3. $A \rightarrow TGS : \underbrace{\{A, TGS, authK, Ta\}K_{tgs}}_{\bullet authTicket}, \underbrace{\{A, T2\}authK}_{\bullet autenticatore\ 1}, B$

4. $TGS \rightarrow A : \underbrace{\{servK, B, Ts, \{A, B, servK, Ts\}K_b\}}_{\bullet servTicket}authK$

- III. SERVIZIO

5. $A \rightarrow B : \underbrace{\{A, B, servK, Ts\}K_b}_{\bullet servTicket}, \underbrace{\{A, T3\}servK}_{\bullet autenticatore\ 2}$

6. $B \rightarrow A : \{T3+1\}servK$



Kerberos – gestione delle chiavi

- AS genera authK al tempo T_a
TGS genera servK al tempo T_s
- Validità
 - di authK (ossia di T_a) in ore, diciamo L_a
 - di servK (ossia di T_s) in minuti, diciamo L_s
 - di un autenticatore (ossia di T_1 , T_2 e T_3) in sec.
- Tgs può generare servK solo qualora sia
$$T_s + L_s \leq T_a + L_a$$
altrimenti problema di **cascata dell'attacco**



Cascade attack

Def. Un attacco ne provoca altri direttamente

- Supponiamo che C abbia violato una chiave di sessione (di autorizzazione) scaduta authK che B condivide con A
- Con semplice decodifica ottiene servK ancora valida se non si impone $T_s + L_s \leq T_a + L_a$

- III. SERVICE

5. **C** $\rightarrow$ B : $\{A, B, \text{servK}, T_s\}_{K_b}, \{A, \text{T3}'\}_{\text{servK}}$

6. B $\rightarrow$ A : $\{T3'+1\}_{\text{servK}}$ (intercettato)

- C può accedere a B per la durata residua di L_s



Discussione

- Replay attack su N-S simmetrico nell'ipotesi che chiavi di sessione **vecchie** siano insicure
 - Vecchio: genericamente, del passato – non esiste temporalità
- Cascade attack su Kerberos nell'ipotesi che chiavi di sessione **scadute** siano insicure
 - Scaduto: specificatamente, il cui intervallo di validità sia scaduto – esiste temporalità



Evoluzione di Kerberos

- Funzionamento inter-realm inefficiente in versione IV, migliorato in V
- Scelta del crittoalgoritmo impossibile in IV (DES); scelta del lifetime limitata in IV
- Sorgenti completi della IV non esportabili
 - Per via dell'implementazione del crittoalgoritmo
- Versione V è uno standard di vastissimo utilizzo (specificato in RFC1510)



Kerberos in pratica

- Installare un KDC sul proprio dominio
- (Parte dei) sorgenti e/o precompilati facilmente reperibili
- Tutte le applicazioni partecipanti devono essere servizi Kerberos

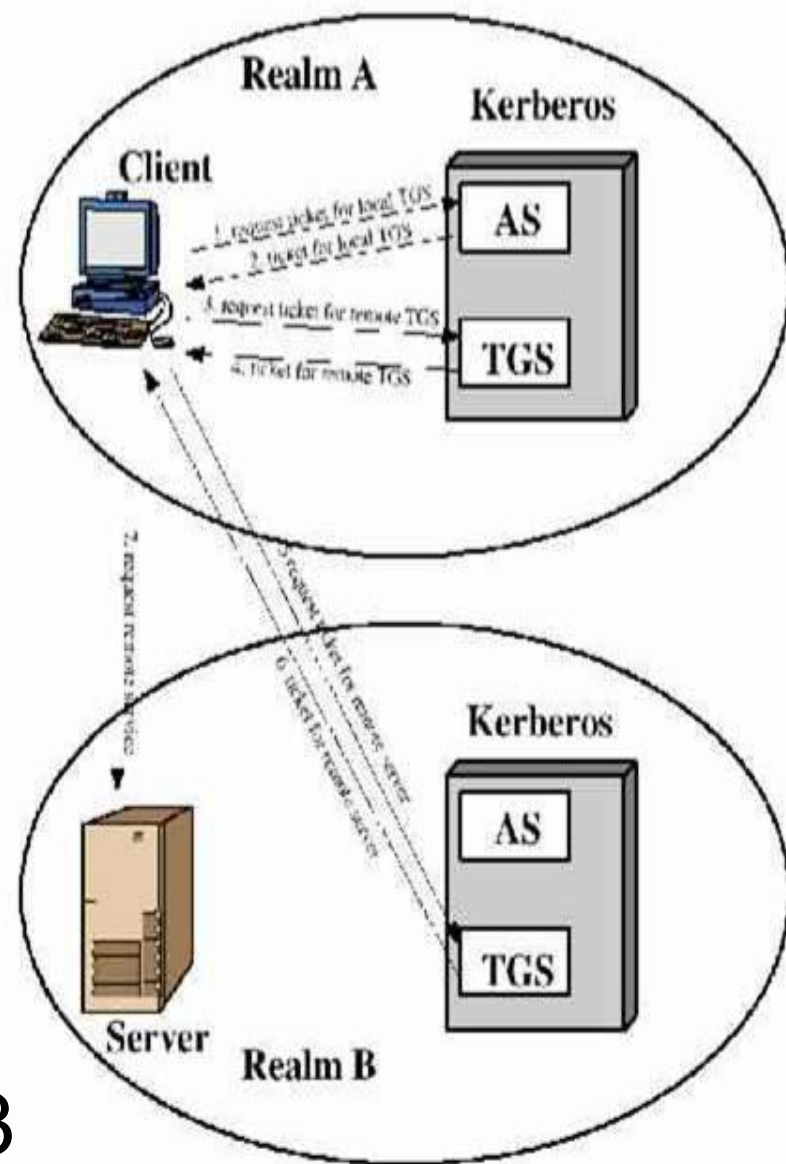


Kerberos

- 3 fasi

1. Autenticazione
2. Autorizzazione
3. Servizio

- **Realm** di Kerberos è un dominio di rete
- Attraversare realm: servono sia la 2 che la 3





Abbiamo visto Kerberos IV

I. AUTENTICAZIONE

1. $A \rightarrow AS : A, TGS, T1$
2. $AS \rightarrow A : \{authK, TGS, Ta, \underbrace{\{A, TGS, authK, Ta\}K_{tgs}}_{authTicket}\}K_a$

II. AUTORIZZAZIONE

3. $A \rightarrow TGS : \underbrace{\{A, TGS, authK, Ta\}K_{tgs}}_{authTicket}, \underbrace{\{A, T2\}authK}_{autenticatore\ 1}, B$
4. $TGS \rightarrow A : \{servK, B, Ts, \underbrace{\{A, B, servK, Ts\}K_b}_{servTicket}\}authK$

III. SERVIZIO

5. $A \rightarrow B : \underbrace{\{A, B, servK, Ts\}K_b}_{servTicket}, \underbrace{\{A, T3\}servK}_{autenticatore\ 2}$
6. $B \rightarrow A : \{T3+1\}servK$



Kerberos IV inter-realm

II. AUTORIZZAZIONE (locale)

3. $A \rightarrow TGS: \{A, TGS, authK, Ta\}K_{tgs}, \{A, T2\}authK, TGSr$

4. $TGS \rightarrow A:$

$\{authKr, TGSr, Tr, \{A, TGSr, authKr, Tr\}K_{tgsr}\}authK$

II'. AUTORIZZAZIONE (remota)

3'. $A \rightarrow TGSr: \{A, TGSr, authKr, Tr\}K_{tgsr}, \{A, T2r\}authKr, Br$

4'. $TGSr \rightarrow A: \{servKr, Br, Ts, \{A, Br, servKr, Ts\}K_{br}\}authKr$

III. SERVIZIO (remoto)

5. $A \rightarrow Br: \{A, Br, servKr, Ts\}K_{br}, \{A, T3\}servKr$

6. $Br \rightarrow A: \{T3+1\}servKr$



Kerberos IV inter-realm

- Se TGS non possedesse K_{tgsr} ?
 - Quel preciso attraversamento sarebbe imposs.
- Ma potrebbe possedere la chiave di un altro TGS, il quale a sua volta...
- Ricercare uno specifico realm diventa lineare nel numero di realm
- Interconnettere n realm è $O(n^2)$



Kerberos in pratica

- Installare un KDC sul proprio dominio
- Tutte le applicazioni partecipanti devono essere servizi Kerberos
- Versione IV ha molti limiti
- Versione V è uno standard di vastissimo utilizzo (specificato in RFC1510)



Versione IV: limiti ambientali

1. **Dipendenza dal crittoalgoritmo**: usa DES, di ristretta esportazione dagli USA
2. **Dipendenza dal protocollo Internet**: solo IP
3. **Rappresentazione dei byte**: il mittente può usare la propria, limitata interoperabilità
4. **Implementazione dei lifetime**: 8 bit per unità di 5min, circa $21\frac{1}{4}$ ore, può non bastare



Versione IV: limiti ambientali

- 5. **No forward di credenziali**: impossibile rimbalzarle da un nodo ad un altro
- 6. **Nomi**: nome e realm di 39 caratteri ciascuno, nome senza il punto
- 7. **Attraversare realm**: fasi 1 e 2 locali, fasi 2 e 3 remote, semplice ma $O(n^2)$ per interconnettere n realm



Versione IV: limiti tecnici

1. **Doppia codifica**: sembra inutile in passi 2, 4
 2. $AS \rightarrow A :$
 $\{\text{authK}, \text{TGS}, \text{Ta}, \{\text{A}, \text{TGS}, \text{authK}, \text{Ta}\}_{\text{Ktgs}}\}_{\text{Ka}}$
 4. $\text{TGS} \rightarrow A :$
 $\{\text{servK}, \text{B}, \text{Ts}, \{\text{A}, \text{B}, \text{servK}, \text{Ts}\}_{\text{Kb}}\}_{\text{authK}}$
2. **Non standard DES**: usa modo PCBC per confidenzialità e integrità, attaccabile



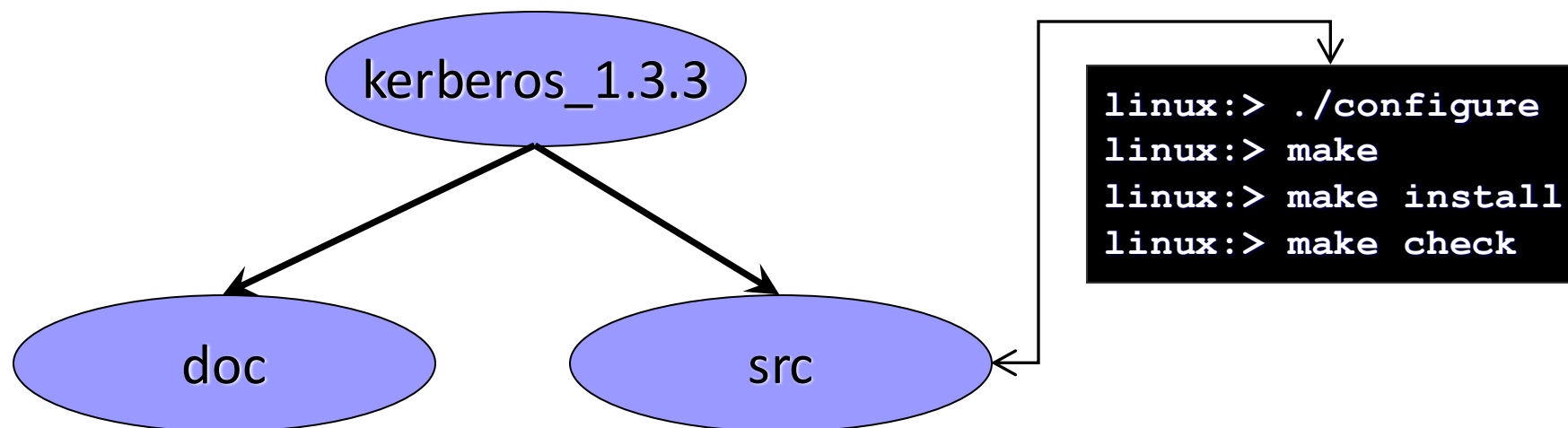
Versione IV: limiti tecnici

3. **Replay degli autenticatori**: manca lista degli autenticatori ancora validi
4. **Attacchi sulla password**: K_a del passo 2 deriva alitmicamente dalla password di A
5. **Replay di chiavi di sessione**: B, ultimata transazione con A, deve ricordarsi di non accettare più materiale criptato con $servK$



Installare Kerberos V

- Una versione recente è `kerberos_1.3.3`

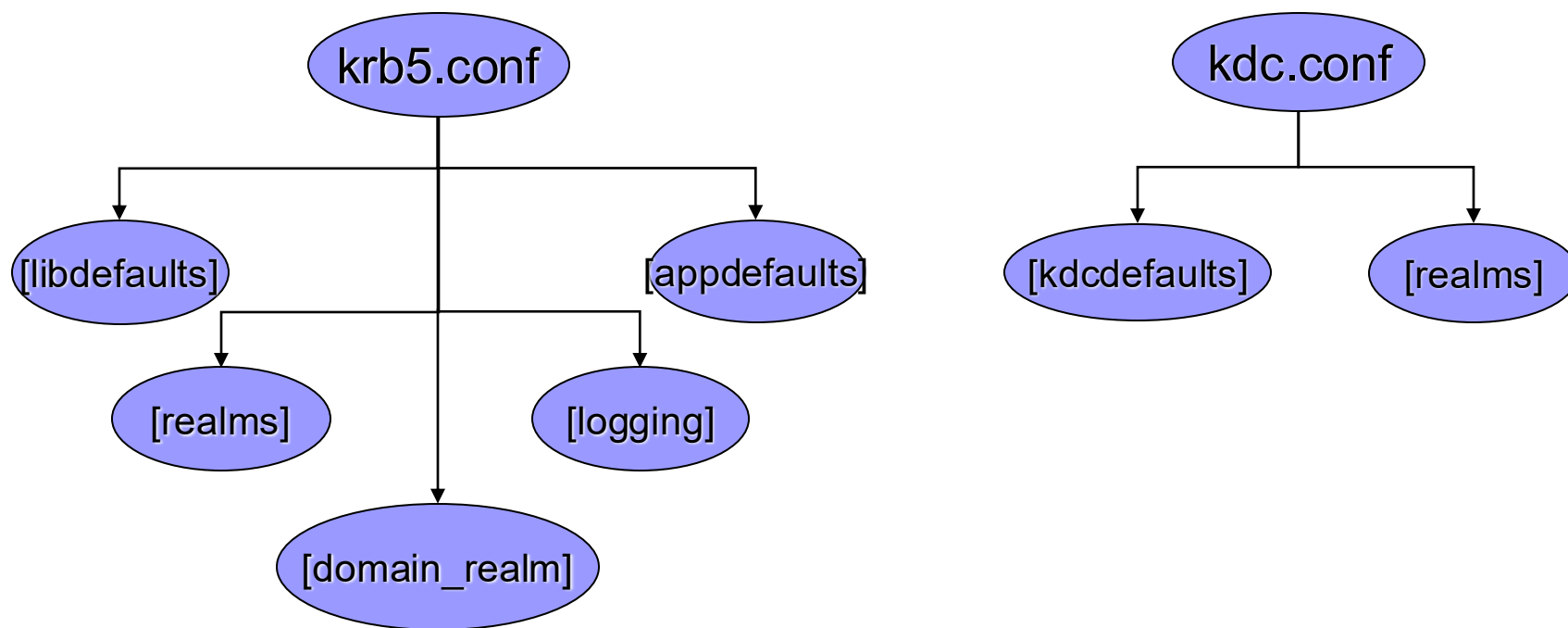


<http://www.redhat.com/docs/manuals/linux/RHL-9-Manual/ref-guide/s1-kerberos-clients.html>



Configurare Kerberos V

■ Struttura dei due file principali





krb5.conf

[libdefaults]: librerie di default

[realms]: informazioni relative ai realm, ossia quali macchine ospitano il KDC (= AS+TGS)

[domain_realm]: tag usato per determinare il realm di appartenenza di una macchina

[logging]: path dei file di log

[appdefaults]: valori di default degli applicativi



kdc.conf

[kdcdefaults]: informazioni relative alle porte che potrà gestire il KDC

[realms]: contiene una sottosezione per ogni realm definito in krb5.conf, ognuna contenente informazioni specifiche per quel realm, per esempio dove trovare il suo KDC



Creare il database del KDC

- Database per il realm definito nei file di configurazione
- Contiene tutti gli utenti del realm, inclusi vari client e servizi (erano A e B)

```
linux:> /usr/local/sbin/kdb5_util create -r MY.REALM -s  
Initializing database '/usr/local/var/krb5kdc/principal' for  
realm 'MY.REALM'. Master key name 'K/M@MY.REALM'
```

```
You will be prompted for the database Master Password.  
It is important that you DON'T FORGET this password.
```

```
Enter KDC database master key:
```

```
Re-enter KDC database master key to verify:
```



Interazione

- Formato nomi: `utente/ruolo@realm`
 - `user1/role2@athena.mit.edu`
 - `admin/admin@MY.REALM`
- Vedere la lista dei certificati in cache
 - `klist`
- Cancellare la cache
 - `kdestroy`
- Richiedere un `authTicket` fresco
 - `kinit`



ACL per il database

- Un file dedicato regola l'amministrazione
- Di default kadm5.acl, stabilito in kdc.conf

```
Kerberos principal      permission      optional target principal
giamp/*@MY.REALM        a m            student1
*/admin@MY.REALM        *              *
```

■ Alcuni permessi

- ☐ a – aggiunta di nuovi utenti
- ☐ d – cancellazione di utenti esistenti
- ☐ m – modifica permessi di utenti esistenti
- ☐ * – carattere jolly



Accesso al database

- Mediante appositi comandi regolati dall'ACL
- Esempio: aggiunta di un amministratore

```
linux:> /usr/local/sbin/kadmin.local addprinc admin/admin@MY.REALM
WARNING: no policy specified for "admin/admin@MY.REALM";
defaulting to no policy.

Enter password for principal admin/admin@MY.REALM:
Re-enter password for principal admin/admin@MY.REALM:
Principal "admin/admin@MY.REALM" created.
```




Lanciare i deamon

```
linux:> /usr/local/sbin/krb5kdc
linux:> /usr/local/sbin/kadmind
linux:> tail /var/log/krb5kdc.log
Jul 27 12 :35 :47 turbot krb5kdc[3187] (info) :
    commencing operation
linux:> tail /var/log/kadmin.log
Jul 27 12 :35 :47 turbot kadmind[3189] (info) :
    starting
```



Esempio

- Un client intende accedere a un server remoto che fornisce un servizio di database usando Kerberos V
- Tutti gli utenti della macchina client che richiedano il servizio vanno registrati nel database del KDC



Server

- Va avviato prima il server
- KDC dà un messaggio di conferma

```
linux:~/Desktop/ProgettoSic/krb5-1.3.3/src/appl/Sicurezza/ap  
plication # ./server  
  
il principal server server@MY.REALM è attivo
```



Client

- Lanciato il client di nome linux, sono richieste le credenziali utente

```
linux:~/Desktop/ProgettoSic/krb5-1.3.3/src/appl/Sicurezza/a
pplication # ./client linux

Inserire username
minmo
Password for minmo@MY.REALM: █
```



Client

- Quindi seguono le 3 fasi del protocollo fino all'accesso al servizio di database

```
Shell - Konsole
Sessione Modifica Visualizza Segnalibri Impostazioni Aiuto

linux:~/Desktop/ProgettoSic/krb5-1.3.3/src/appl/Sicurezza/ap
plication # ./client linux

Inserire username
mimmo
Password for mimmo@MY.REALM:

Ho ottenuto il ticket per mimmo@MY.REALM

Connesso all'host server

Ho ricevuto la mutua autenticazione del principal server con
tattato server@MY.REALM

Comandi:
1) leggi i dati di un utente
2) inserisci nuovo utente
3) modifica utente
4) cancella utente
0) uscita
```



Server

- Il KDC notifica al server la presenza del client

```
linux:~/Desktop/ProgettoSic/krb5-1.3.3/src/appl/Sicurezza/ap  
plication # ./server  
  
il principal server server@MY.REALM è attivo  
  
Principal mimmo@MY.REALM connesso  
█
```



Client

- Il client decide di terminare

```
Shell - Konsole
Sessione Modifica Visualizza Segnalibri Impostazioni Aiuto

Comandi:
1) leggi i dati di un utente
2) inserisci nuovo utente
3) modifica utente
4) cancella utente
0) uscita
1
Inserisci chiave          marros

DATI UTENTE
  nome:marcoro
  cognome:Rossi
  luogo_di_nascita:Salerno
  data_di_nascita:30/06/1982
  residenza:Cava(SA)
  indirizzo:via_Veneto_190
  cittadinanza:ita

Comandi:
1) leggi i dati di un utente
2) inserisci nuovo utente
3) modifica utente
4) cancella utente
0) uscita
0
      Grazie per aver scelto il nostro programma

linux:~/Desktop/ProgettoSic/krb5-1.3.3/src/appl/Sicurezza/ap
plication #
```



Server

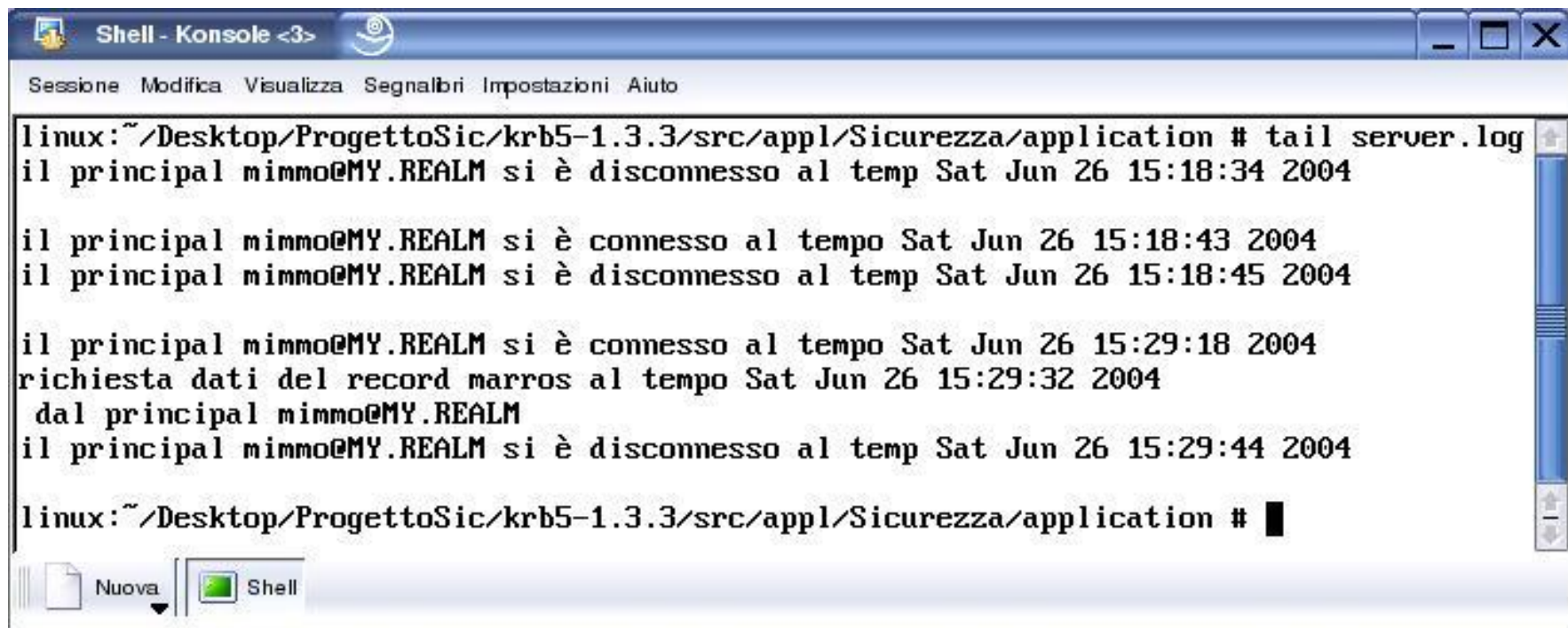
- Il KDC notifica al server tutte le connessioni

```
linux:~/Desktop/ProgettoSic/krb5-1.3.3/src/appl/Sicurezza/ap  
plication # ./server  
  
il principal server server@MY.REALM è attivo  
  
Principal mimmo@MY.REALM connesso  
il principal mimmo@MY.REALM ha terminato  
Principal armando@MY.REALM connesso  
il principal armando@MY.REALM ha terminato  
█
```




File log

- Registrando il tutto in un apposito file di log



```
linux:~/Desktop/ProgettoSic/krb5-1.3.3/src/appl/Sicurezza/application # tail server.log
il principal mimmo@MY.REALM si è disconnesso al temp Sat Jun 26 15:18:34 2004

il principal mimmo@MY.REALM si è connesso al tempo Sat Jun 26 15:18:43 2004
il principal mimmo@MY.REALM si è disconnesso al temp Sat Jun 26 15:18:45 2004

il principal mimmo@MY.REALM si è connesso al tempo Sat Jun 26 15:29:18 2004
richiesta dati del record marros al tempo Sat Jun 26 15:29:32 2004
dal principal mimmo@MY.REALM
il principal mimmo@MY.REALM si è disconnesso al temp Sat Jun 26 15:29:44 2004

linux:~/Desktop/ProgettoSic/krb5-1.3.3/src/appl/Sicurezza/application #
```



Versione V: crittoalgoritmi

- Implementati come moduli SW facilmente rimpiazzabili
- Crittotesti e chiavi etichettati col nome
- Usa DES in modo standard CBC
 - A scopo d'integrità aggiunge MAC prima della codifica



Versione V: protocollo e byte

- Supporto per vari protocolli di rete
 - Un tag accompagna ciascun indirizzo di rete
- Specificata rappresentazione dei byte cui uniformarsi



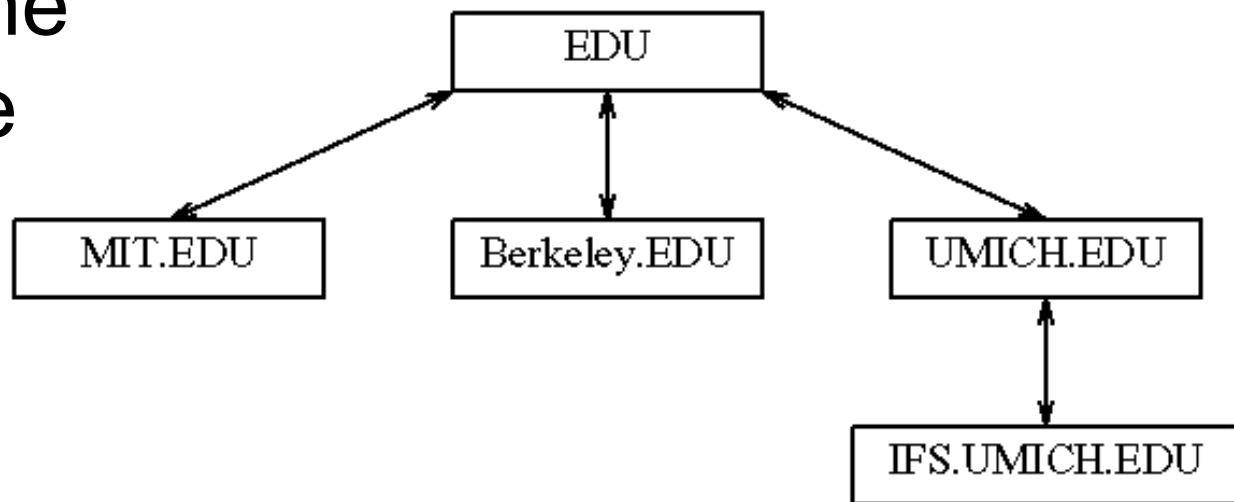
Versione V: forward di credenziali

- Un flag stabilisce se un authTicket è rimbalzabile
- Il client A presenta l'authTicket al TGS dal suo nuovo indirizzo
 3. $A' \rightarrow TGS: \{A, TGS, authK, Ta\}K_{tgs}, \{A', T2\}authK, B$
- TGS replica al nuovo indirizzo
 4. $TGS \rightarrow A': \{servK, B, Ts, \{A', B, servK, Ts\}K_b\}authK$



Versione V: inter-realm

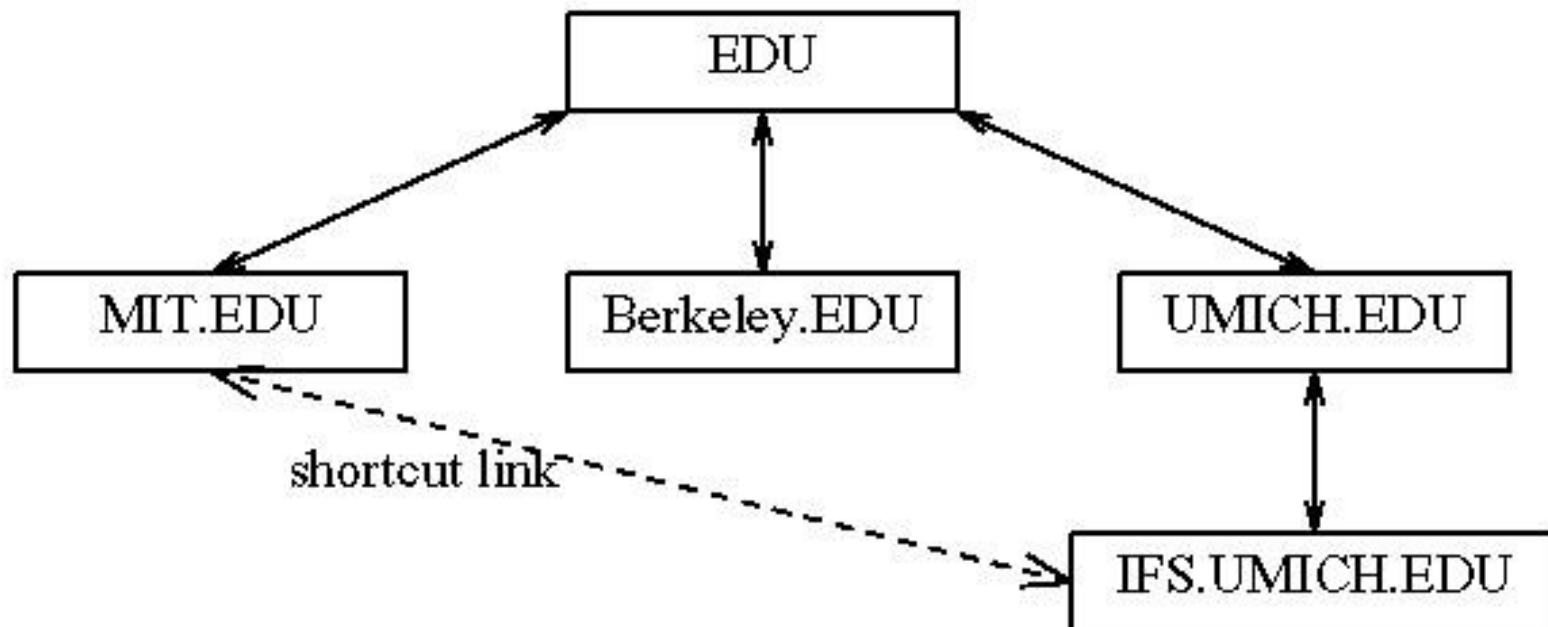
- Realm organizzati ad albero (bilanciato): complessità logaritmica nel numero di realm per connetterne due
- Connessione di n realm è $O(n \lg n)$





Versione V: inter-realm

- I percorsi sono ottimizzabili





Versione V: no doppia codifica

- Verifica formale ne conferma l'inutilità

2. $AS \rightarrow A : \{\text{authK}, \text{TGS}, \text{Ta}\}_{K_a},$
 $\{A, \text{TGS}, \text{authK}, \text{Ta}\}_{K_{\text{tgs}}}$

4. $\text{TGS} \rightarrow A : \{\text{servK}, B, \text{Ts}\}_{\text{authK}},$
 $\{A, B, \text{servK}, \text{Ts}\}_{K_b}$



Ticket rinnovabile

- Nuova caratteristica della versione V
 - Un apposito flag stabilisce se rinnovabile
 - Ha due lifetime: L_1 ed L_2 , essendo $L_1 < L_2$
 - Scadenza effettiva dopo L_1
 - Rinnovabile anche entro L_1 ma comunque non oltre la fine di L_2
- + KDC non rinnova ticket riportati rubati



Versione V: password

- Ka ottenuta da codifica di password di A insieme al realm
 - ☐ Ka non è quindi la stessa su tutti i realm
 - ☐ Dovrebbe proteggere i realm l'uno dall'altro?
 - ☐ Perché non usare salt per aumentare la ridondanza della chiave?
- Supporto per one-time password prodotte per esempio con smart token



Però

- Rimane il rischio di dictionary attack su K_a
 - Bisogno di memorizzare K_a in maniera sicura
 - Limite intrinseco: K_a è hash di password di A
- K_a dovrebbe essere random
- Idea: usare smartcard



Kerberos V con smartcard

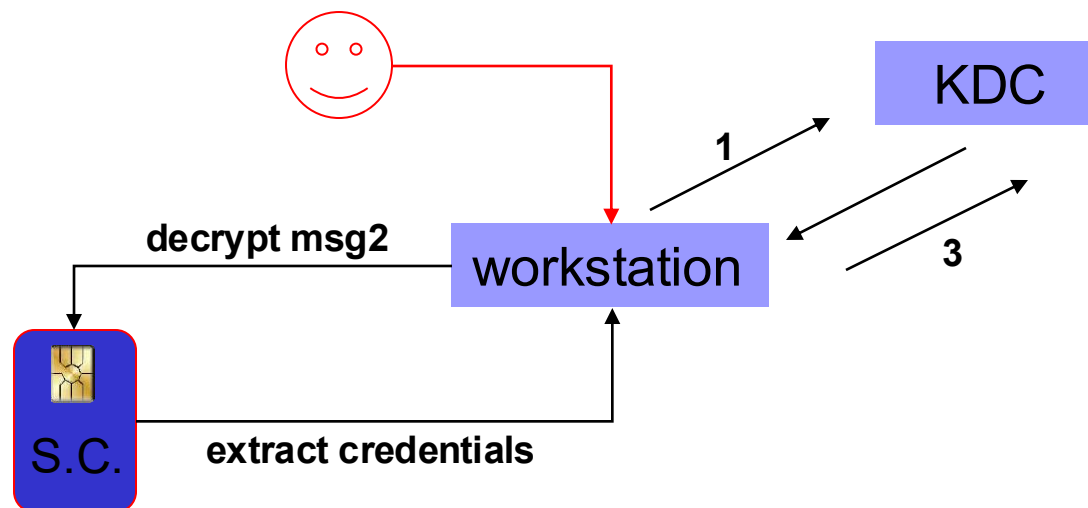
- Peter Honeyman et al, 1998

We implemented the smartcard integrated Kerberos protocol described in Section 3. We now detail the modifications we made to the Kerberos library, the DES library, and the Kerberos client.

TGT decryption is implemented with a STARCOS version 2.1 smartcard from Giesecke & Devrient. This card offers superior performance by providing native *cipher block chaining* (CBC) for long messages. (A Kerberos V5 TGT is over 200 bytes long.) The development platform is OpenBSD-2.2 on Pentium 133MHz PC. The code base is Kerberos version 1.0.5 released by MIT.



Disegno



- Ka random invece che hash di password
- Salvata in smartcard (+KDC), mai emessa
- Decodifica del passo 2 in smartcard
- AuthTicket e authKey resi alla workstation



Prestazioni (!)

- Interazione con smartcard ca. 1.5 secondi
 - Decodifica ne richiede il 70%
- Smartcard: 3.5MHz, data path a 8bit



Conseguenze

- Formalmente risolto dictionary attack su Ka
- Problemi aperti
 - ☐ Smartcard devono essere protette da PIN
 - ☐ Mezzo fra workstation e smartcard sicuro?
 - ☐ Credenziali passate alla cache della workstation a rischio
- Risolvibili con tecnologia attuale?
 - ☐ Intera sessione su smartcard?



Approfondimenti

- Neuman e Ts'O: **The Evolution of the Kerberos Authentication System**
- Installare Kerberos V su Ubuntu... c'è qualche problemino...
- Kerberos vs. Radius